
Brain Signals to Sentences Using Deep Learning

Kriti Chakrabarty
kchak@unc.edu

Rachel Kudzin
rachieg2@unc.edu

Sean Shen
sjshen@unc.edu

Barrett White
barrlwhi@unc.edu

Abstract

This paper aims to explore new deep learning methods to decipher brain activity (electroencephalography (EEG) signals) into coherent phrases via predicting phonemes and then words. We expanded the research conducted for the Brain-to-Text 2024 Kaggle competition [4] by introducing transformer architectures, including the use of pre-trained models. Our architectures are based on the existing literature regarding EEG signal transformations and categorizations. We attempt to re-implement the baseline competition winning model (GRU) and design a pure encoder-decoder, an untrained CNN (EEGNet) and pre-trained transformer (T5), and a pre-trained CNN (ShuffleNet) and pre-trained transformer (T5). With limited compute as students, we did not see the advantages that may exist with using these pre-trained, open-source models. During our experimentation, the original GRU architecture, finetuned on the 2025 data, performed the best, with a final word error rate (WER) of 10.8%. A lightweight transformer architecture also performed decently, with a WER of 13.7%. From our experimentation on four model types, it appears that the original 2024 competition winner still performs the best on this task in 2025. The complete code needed to run our experimentation and replicate our results is available on Github.

1 Introduction

Our team has attempted to create a model for the open Kaggle competition Brain-to-Text 2025 [3]. The competition's goal is to develop an algorithm to transform electroencephalography (EEG) brain activity into words, thus restoring communication for those who have lost the ability to move and speak. During this process, we developed models that can accurately decode speech from complex neural patterns and signals reasonably well, which involved mapping a high-dimensional, variable-length time series of neural activity data to text. We re-implemented the former competition's model and created three transformer-based architectures with varying success.

2 Related Works

The Brain-to-Text 2025 competition follows a similar one from 2024. The authors outline the results in [4]. This architecture by Card et al. uses a recurrent neural network for neural decoding and a large-vocabulary language model for text generation, which has demonstrated that a brain-computer interface can decode the attempted speech of a man with ALS and dysarthria into words with 97.5% accuracy. The modelers used an RNN-Based Neural Decoder with an N-Gram Model. The decoder's network has a linear day-specific layer to capture the non-stationarity of neural data, five layers of gated recurrent unit architecture with 512 units per layer, and a dense output that produces probabilities of 41 classes (mostly phonemes). The authors trained a new RNN each research session with the research participant, where new data collection and evaluation of the model occurred. The N-Gram language model takes sequences of phoneme probabilities as input, and outputs the most likely sequence of words with a 125,000-word vocabulary trained on OpenWebText2. The model starts by predicting up to 100 of the most likely sequences of words before ranking those sequences to identify

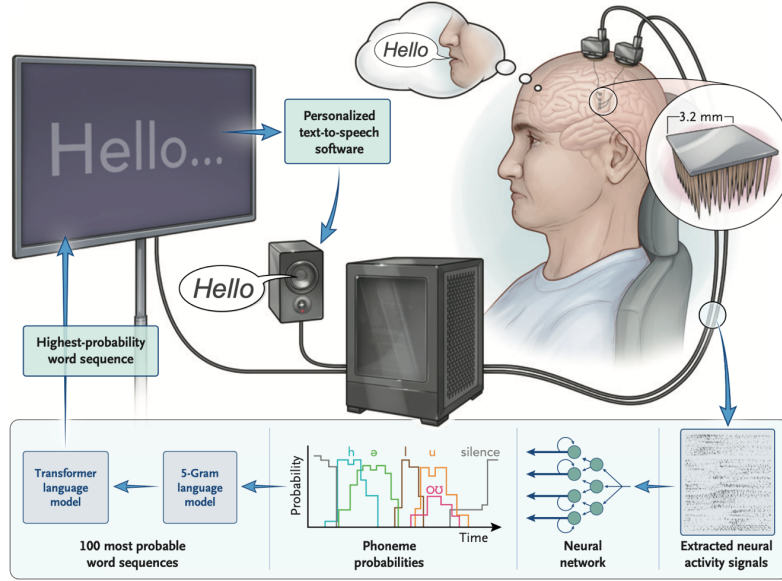


Figure 1: Illustration of Brain-to-text [4]

the best option. The authors also suggest that transformer architectures may show improvements over their methods for the 2025 competition.

Several groups have attempted this problem, both during these competitions and outside of them. In general, key issues with this type of data include its high dimensionality, but usually, there is a relatively small corpus due to limited data collection. Chen et al. [5] mapped the high dimensions to lower dimensions with a Hierarchical Gated Recurrent Unit with Masked Residual Attention Mechanism (HGRU-MRAM) model. The HGRU focuses on local patterns, including temporal features, while the MRAM records global information used to finetune BART. At the time of publication, the resulting model outperformed other existing methods.

Benster et al. [2] present a cross-modal silent-speech system that learns a shared representation between sensor signals and audio with contrastive training, then boosts decoding using a large language model for rescoring. In broad terms, it mixes multimodal alignment, self-contrastive objectives, and an LM decoder to get more out of limited labeled data, yielding substantially lower error than prior approaches.

Zhang and Li [11] discuss attempts to use Transformer architecture in their survey paper. Many of these models include transformer encoders and decoders only, while others include hybrids. An interesting hybrid approach involves using a CNN to extract signals from the high-dimensional space, followed by an encoder-decoder infrastructure on the lower-dimensional results. Wu et al. also provide details of a modeling pipeline that include tips on every step, from data cleaning to modeling when using transformers in NLP.

Lévy et al. [9] found success with a three-model architecture, trained on the Spanish language. The first module is a convolution layer, which embeds temporal and spatial information from the EEG data. The first layer of this module is a spatial attention mechanism, which moves the high dimensional data into a lower dimensional space. Then, a 8-layer CNN employs dropout and GELU activation. The next module is a transformer that ends with a linear layer. The final module is a 9-gram character language model, which achieves an average of 32% character-error-rate.

Jayalath et al. [7] present a non-invasive brain-to-text pipeline that pairs a simple neural decoder with a large language model to clean up and complete predictions, and demonstrate that scaling data and vocabulary handling can push performance beyond prior non-invasive baselines. They turn brain-signal classification into text generation by using an LM to rescore candidate outputs and fill in missing words, and train across datasets to leverage more diverse data, yielding markedly better accuracy than earlier work.

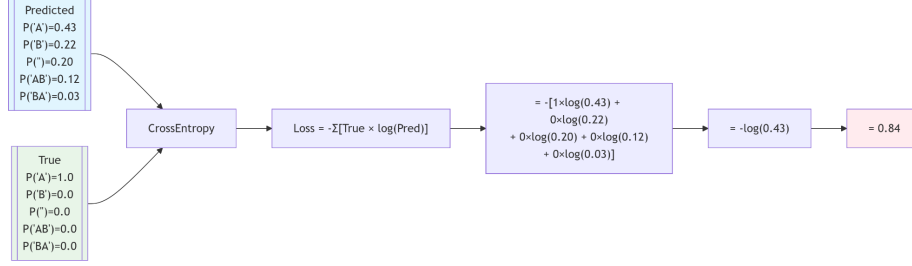


Figure 2: Distributions Compared using Cross Entropy

There has also been work using pre-trained models, allowing the final classification layer to be modified during training. Sadiq et al. [10] explore the process of transforming EEG signals into pseudo-images using multiscale principle component analysis (MSPCA) and then continuous wavelet transformations (CWT). By feeding these pseudo-images into multiple pre-trained CNNs, they discover that ShuffleNet [12] produces the best results.

3 Dataset and Evaluation Metric

The UC Davis Neuroprosthetics Lab provided the competition dataset which contains 10,948 sentences spoken by a single participant. The data has both the sentences that the participant was attempting to say and the corresponding time series of neural spiking activity from the 256 microelectrodes in the speech motor cortex. There are different sentence corpora, such that different types of speech (conversational, web text, high frequency words) are all included. We held out 10% of the data for our test set. Additionally, we used approximately 20% of the training data as a training subset for the non-baseline models, to conserve compute power. We evaluated the decoding performance using the held-out test sentences with the word error rate, the competition authors' preferred metric. The word error rate is defined as the number of substitutions, insertions, and deletions of words needed to make the decoded sentence match the true sentence, divided by the total number of words in the true sentence. The aggregated word error rate across all sentences is equal to the summed edit distance over all sentences, divided by the total number of words across all the true sentences.

4 Methodology

Based on the various techniques used in the literature, we decided to implement four model architectures, as follows:

1. Re-implementation of the 2024 Brain-to-Text Model as the baseline
2. Pure encoder/decoder transformer architecture
3. Untrained CNN (EEGNet [8]) + pre-trained transformer (FLAN-T5-small [6])
4. Pre-trained CNN (ShuffleNet [12]) + pre-trained transformer (FLAN-T5-small [6])

After building these models, we tested their effectiveness using the phoneme error rate. For models with reasonable phoneme error rates ($\leq 25\%$), we then put their outputted phonemes into the N-Gram language model from the original paper to get the final word error rate [3].

The N-gram language models take sequences of phoneme probabilities as input and output the most likely sequence of words. We specifically used the pretrained 3-gram model because the 5-gram requires much more space and RAM on the Longleaf which may take an unreasonable amount of time to queue. The 3-gram model was trained on the OpenWebText2 corpus to learn the correct way to arrange words. Then the 3-gram model utilizes the CMU Pronouncing Dictionary to define a fixed vocabulary of 125,000 words and their corresponding phonetic spellings, enabling the decoder to map raw phoneme sequences into valid text. To use the 3-gram model, it is compiled into a standalone TLG.fst graph that encodes transition probabilities for sequences of up to 3 words. There are 2

parameters associated with the language model decoding, the `acoustic_scale` and `blank_penalty`. The `acoustic_scale` weighs the trust between received phoneme probabilities and grammar rules. The `blank_penalty` penalizes the blank labels predicted during the decoding process.

4.1 Brain-to-Text Model: GRU Decoder

The baseline Brain-to-Text Model is a recurrent neural network that uses the gated recurrent unit architecture to translate high-dimensional neural activity into phoneme probabilities. The neural activities were recorded over many days. To address the non-stationarity of neural signals across recording days, the network has a session-specific input alignment layer that applies a linear transformation and Softsign activation to adapt to the changes in placement of electrodes. After the session-alignment, the input features are aggregated using a sliding window that concatenates 14 time steps with a stride of 4. The aggregations of time steps increase the computation efficiency and capture the long-range temporal dependencies across the speech sequence. Those concatenations/aggregations are then processed by 5-layer unidirectional GRU with 768 hidden units. Finally, a linear classifier is used to output the logits for the 41 phoneme classes. The architecture is illustrated by 7.

To adapt this pre-trained baseline to the new data this year, we finetuned the model’s parameters in two stages to prevent catastrophic forgetting. First, we unfroze the session-specific input alignment layers for the new sessions, where the new session-specific input parameters were initialized using the mean of the pretrained weights. There was no significant reduction in the test phoneme error rate ($0.1020 \rightarrow 0.1010$). For the second stage of finetuning, all of the parameters were unfrozen and trained using a reduced learning rate of 1×10^{-5} . Since there was significant data imbalance between the historical data and the new sessions, we used a biased sampling for making batches during training with a 90% probability of choosing a new session and a 10% probability of choosing an old session. There was, again, no significant reduction in the test phoneme error rate ($0.1010 \rightarrow 0.1005$).

4.2 Pure Encoder-Transformer

This model used a small transformer encoder designed as a direct neural decoder. Each trial first passes through a day specific linear layer with Softsign activation and dropout, providing a simple way to adjust to session differences. The output is projected to a 224 dimensional embedding and combined with sinusoidal positional encodings.

We then apply a 2 layer transformer encoder with 4 attention heads, GELU activations, and dropout. A linear output layer produces phoneme logits for each time step. The model is trained using the standard CTC loss, allowing alignment between neural activity and phoneme sequences without requiring explicit segmentation.

A drawing of this architecture is available in 4.

4.3 EEGNet and T5

For our trial with an untrained CNN and pre-trained transformer, we picked EEGNet [8] and FLAN-T5-Small [6]. EEGNet is a CNN architecture specifically designed for EEG signals. In particular, EEGNet shows reasonable performance at extracting features even with limited training data [8]. We also chose T5 as our pre-trained model because of its wide applications in language modeling and strong performance on benchmark NLP tasks [6]. In this case, T5 brings in the feature output from EEGNet, which is then fed through feature projection and embedding layers into T5. We freeze all layers in T5 except for the final classification layer. Due to compute limitations, we use FLAN-T5-small.

A schematic of the architecture of this model is available in 5.

4.4 ShuffleNet and T5

We also wanted to test a CNN and transformer model where we utilized pretrained options for both pieces. We use ShuffleNet for our pre-trained CNN, as Sadiq et al. found that this model led to the best performance for EEG data [10], due to its ability to preserve the time-frequency structure of the signals. ShuffleNet requires specific input dimensions that match normal image requirements; that is, the inputs must be RGB images of shape $(3 \times H \times W)$, with H and W being at least 224 pixels

```

Session: t15.2023.08.13, Block: 9, Trial: 2
Sentence label:    Not for the job I have now.
True sequence:     N AA T | F AO R | DH AH | JH AA B | AY | HH AE V | N AW |
Predicted Sequence: N AA T | F AO R | DH AH | JH AA B | AY | HH AE V | N AW |

Session: t15.2023.08.13, Block: 9, Trial: 3
Sentence label:    It's really not too difficult.
True sequence:     IH T S | R IH L IY | N AA T | T UM | D IH F AH K AH L T |
Predicted Sequence: IH T S | R IH L IY | N AA T | T UM | D IH F AH K AH L

Session: t15.2023.08.13, Block: 9, Trial: 4
Sentence label:    Collisions should never happen.
True sequence:     K AH L IH ZH AH N Z | SH UH D | N EH V ER | HH AE P AH N |
Predicted Sequence: K AH L IH ZH AH N Z | SH UH D | N EH V ER | HH AE P AH N |

```

Figure 3: Phoneme Prediction Example: Baseline GRU

[12]. Therefore, we could not simply input the EEG signal data into ShuffleNet. Using the method from [10], we first ran the EEG data through an MSPCA method, followed by a mean calculation for all nodes in the signal to create one overall signal. This signal was then fed through a CWT, using a Morse window as discussed in [10]. This signal was then transformed into an RGB image and transformed as required by ShuffleNet: resizing and transforming with the required standard deviations and means. After this preprocessing, the EEG signals could be fed into ShuffleNet.

After the CNN, we use FLAN-T5-small for the same reasons discussed above. We freeze all the layers in the CNN and T5 models except the final T5 classifier output, which we train during finetuning.

A figure of this architecture can be found in 6.

5 Results and Discussion

We began by investigating the PER for each model. With limited compute, we were only able to train for a few days via Google Colab or Longleaf, as is further discussed in 5.1. The PER results are displayed in 1.

Table 1: Phoneme Error Rate Results

Model	PER
Baseline GRU	0.1020
Baseline GRU Finetuned	0.1005
Pure Encoder-Transformer	0.2189
EEGNet and T5	0.9914
ShuffleNet and T5	0.9653

The most interesting result of the PER experiments was the apparent failure of the pre-trained models, with an error rate that actually *increased* with training. After nearly 12 hours of training and 800+ batches, the ShuffleNet and T5 model, for example, showed a lowered training loss ($133.476 \rightarrow 92.569$) and validation loss ($131.32 \rightarrow 100.23$). However, the PER appeared to diverge, with the error increasing to 100% after these batches. Our hypothesis for this confusing result is that these pretrained models were simply too big for our limited compute resources. The ShuffleNet and T5 model, even after freezing nearly all pretrained layers, had 16,475,177 trainable parameters. We believe that with unlimited compute and time, these models would have begun to decrease PER and improve overall performance, but we were unable to test this theory with the resources available to us as students.

As previously mentioned, we only used models with PER of less than 25% for investigating the WER. Thus, we ran the language model for both the original baseline GRU and the fine-tuned version, as well as the Pure Encoder-Transformer model. The results for the final WER for these models is shown in 2.

Table 2: Word Error Rate Results

Model	WER
Baseline GRU	0.108
Baseline GRU Finetuned	0.108
Pure Encoder-Transformer	0.137

Perhaps surprisingly, our results show that the original method from the 2024 competition continues to perform the best for this type of data and task.

5.1 Challenges and Limitations

The team ran into several challenges during the development of this model. Computation power was the predominant concern due to the large size of the data. The initial challenges of working with the pretrained N-gram language model involved compiling a custom library `lm_decoder` using CMake on UNC Longleaf. The building process encountered dependency shadowing, where CMake repeatedly used the incompatible libraries provided by the cluster’s Anaconda module rather than the local, custom-build conda environment. We resolved this problem by modifying `CMakeLists.txt` to explicitly keep the system from looking into Longleaf’s Anaconda and look into the location of the correct compiled static libraries (`gflags`, `glog`) in our conda environment. We also encountered the problem of the symlinks not working correctly on Longleaf. We solved that problem by rewriting the build configuration to use physical paths so that the compiler could locate the necessary headers.

Setting up a Python runtime was difficult due to conflicting versions of the libraries used in the original study. The language model’s `lm_decoder` was compiled with `numpy 1.24.4`, but the phoneme decoder setup script they provided was using `numpy 2.1.2`. The baseline phoneme decoder checkpoint provided by the original study worked fine as it was saved using `numpy 1.x`. However, the finetuned checkpoint failed to load because the training process used `numpy 2.1.2` as directed by the set up script. The finetuned checkpoints were incompatible with the language model. We solved this problem by loading the finetuned checkpoints and resaving them using the compatible `numpy 1.24.4` format.

The original codebase used a distributed microservice architecture that relied on Redis to link the RNN Decoder and the language model server. This architecture was tricky to operate in an HPC environment such as Longleaf. We eliminated Redis from our pipeline by importing the compiled `lm_decoder` directly into the evaluation script.

6 Future Work

Looking ahead, there are opportunities to improve the models detailed by bolstering accuracy, improving data augmentation and further fine-tuning the existing Language Model to handle the size of the dataset more effectively.

With additional time and resources, we could continue to improve the pre-trained transformer models. A potential area of improvement would certainly be longer training time, while another could be better signal to image transformations before using a CNN like EEGNet or ShuffleNet. Our signal-to-image pipeline followed the general suggestions in [10], but we did not fully explore the possibilities for preserving as much signal as possible through multiple rounds of MSPCA versus a matrix average across all nodes. As the original experiment shows, much of the improvement in this modeling problem may come from original data augmentation and transformation rather than model architectures [4].

References

- [1] J. Alammari. The illustrated transformer. *The Illustrated Transformer–Jay Alammari–Visualizing Machine Learning One Concept at a Time*, 27(1), 2018.
- [2] T. Benster, G. Wilson, R. Elisha, F. R. Willett, and S. Druckmann. A Cross-Modal Approach to Silent Speech with LLM-Enhanced Recognition, Mar. 2024.

- [3] N. Card, M. Wairagkar, C. Iacobacci, X. Hou, T. Singer-Clark, F. R. Willett, E. M. Kunz, C. Fan, M. V. Nia, D. R. Deo, A. Srinivasan, E. Y. Choi, M. F. Glasser, L. R. Hochberg, J. M. Henderson, K. Shahlaie, S. D. Stavisky, and D. M. Brandman. Brain-to-text '25, 2025.
- [4] Card Nicholas S., Wairagkar Maitreyee, Iacobacci Carrina, Hou Xianda, Singer-Clark Tyler, Willett Francis R., Kunz Erin M., Fan Chaofei, Vahdati Nia Maryam, Deo Darrel R., Srinivasan Aparna, Choi Eun Young, Glasser Matthew F., Hochberg Leigh R., Henderson Jaimie M., Shahlaie Kiarash, Stavisky Sergey D., and Brandman David M. An Accurate and Rapidly Calibrating Speech Neuroprosthesis. *New England Journal of Medicine*, 391(7):609–618, Aug. 2024.
- [5] Q. Chen, Y. Wang, F. Wang, D. Sun, and Q. Li. Decoding text from electroencephalography signals: A novel Hierarchical Gated Recurrent Unit with Masked Residual Attention Mechanism. *Engineering Applications of Artificial Intelligence*, 139:109615, Jan. 2025.
- [6] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, S. Narang, G. Mishra, A. Yu, V. Zhao, Y. Huang, A. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei. Scaling instruction-finetuned language models, 2022.
- [7] D. Jayalath, G. Landau, and O. P. Jones. Unlocking non-invasive brain-to-text. *arXiv preprint arXiv:2505.13446*, 2025.
- [8] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung, and B. J. Lance. Eegnet: a compact convolutional neural network for eeg-based brain–computer interfaces. *Journal of Neural Engineering*, 15(5):056013, July 2018.
- [9] J. Lévy, M. Zhang, S. Pinet, J. Rapin, H. Banville, S. d’Ascoli, and J.-R. King. Brain-to-text decoding: A non-invasive approach via typing. *arXiv preprint arXiv:2502.17480*, 2025.
- [10] M. T. Sadiq, M. Z. Aziz, A. Almogren, A. Yousaf, S. Siuly, and A. U. Rehman. Exploiting pretrained CNN models for the development of an EEG-based robust BCI framework. 143:105242.
- [11] H. Zhang and H. Li. Transformer-based EEG Decoding: A Survey. *arXiv preprint arXiv:2507.02320*, 2025.
- [12] X. Zhang, X. Zhou, M. Lin, and J. Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices, 2017.

A Model Architectures

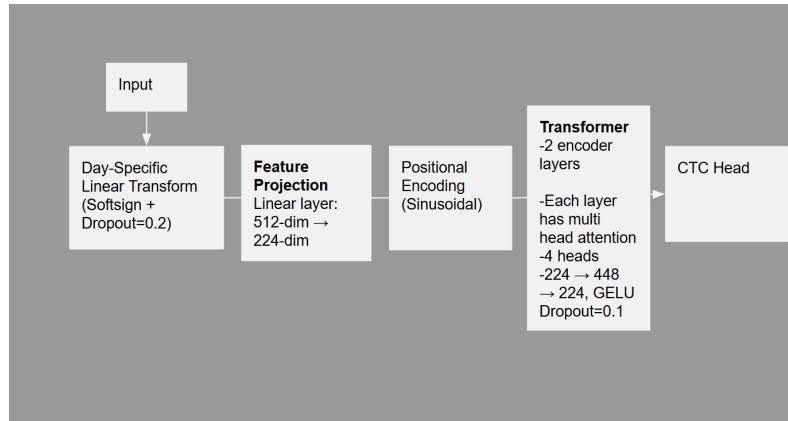


Figure 4: Encoder-Transformer Architecture

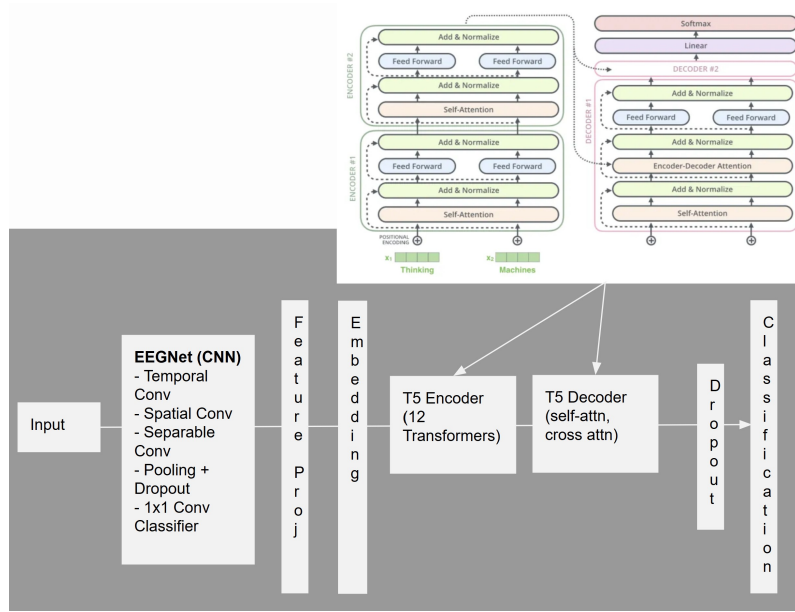


Figure 5: EEGNet and T5 Architecture [1]

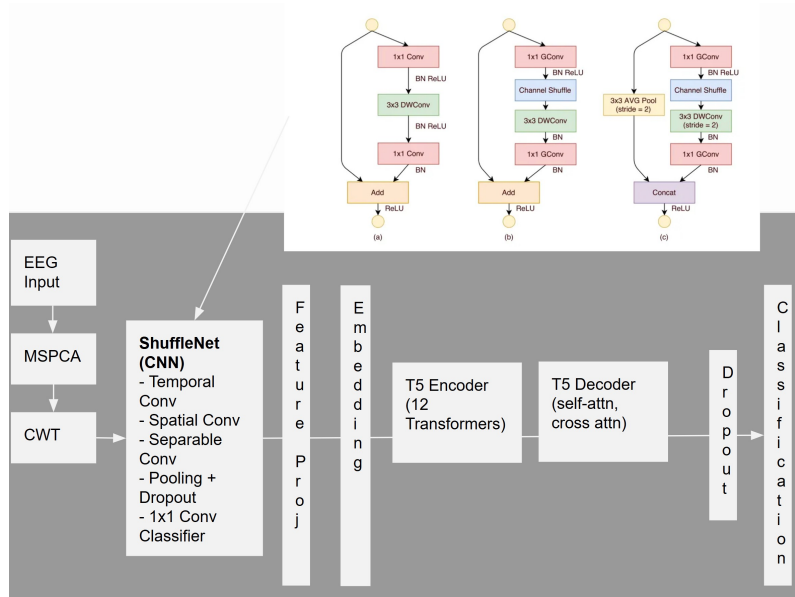


Figure 6: ShuffleNet and T5 Architecture [12]

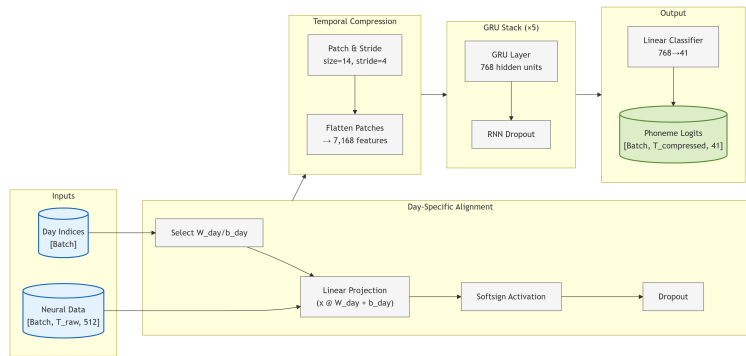


Figure 7: Brain-to-Text GRU Decoder [3]